

TL;DR: архитектурные решения — это выбор под задачу

Table of contents

TL;DR: архитектурные решения — это выбор под задачу	1
Зачем это прикладному инженеру	1
Типовые возражения	1
Итог	2

TL;DR: архитектурные решения — это выбор под задачу

За модуль мы разобрали:

- **Компоненты, слои, SOLID, DI** — как физически организован код и кто от кого зависит, почему без внедрения зависимостей тесты превращаются в мучение, а замена инфраструктуры — в переписывание половины системы.
- **Архитектурные стили** — Clean Architecture, framework-based (Rails, Django), Actor Model (Erlang, Akka), Data-Oriented Design. Каждый оптимизирован под свой класс задач: стартап, телеком с пятью девятками, игровой движок.
- **Сетевая модель OSI и контракты API** — от напряжения на проводе до OpenAPI/proto/GraphQL SDL. Каждый уровень — отдельный договор; верхние не знают, как работают нижние.
- **Прикладные протоколы** — REST, gRPC, GraphQL, WebSocket. Разные по модели (запрос-ответ, RPC, графовая выборка, двусторонний поток), производительности и цене эксплуатации.
- **Синхронные и асинхронные модели** — прямой вызов с ожиданием против публикации в брокер. У синхронного вызова есть свой контур устойчивости: `timeout`, `retry` с `backoff/jitter`, `idempotency`, `circuit breaker`, `deadline propagation`.

Зачем это прикладному инженеру

Модуль — про **выбор под ограничения**, а не про «правильный способ». REST не лучше gRPC, Clean Architecture не лучше Rails, синхронный вызов не лучше асинхронного. У каждого — свой класс задач и своя цена.

Полезно ли зубрить все семь уровней OSI? Сомнительно. Но понимать, что TLS отвечает за шифрование, TCP — за надёжную доставку, а HTTP — за прикладной смысл, бесценно: когда API «не работает», вы сузите причину за минуту, а не за час.

Типовые возражения

«Все вокруг пишут микросервисы, давайте порежем монолит» звучит современно, но каждый новый синхронный вызов приносит с собой `timeout`, `retry`, идемпотентность, `circuit breaker` и передачу дедлайна. Поэтому перед дроблением сначала проверяют,

действительно ли нужны командная автономия, независимый деплой и изоляция отказов, или достаточно нормальной модульности внутри монолита.

«Возьмём REST, потому что все так делают» так же плох как любой выбор по привычке. Для публичного API обычно удобен REST с OpenAPI, для плотного внутреннего трафика часто лучше gRPC, для гибкой выборки полей клиентом может подойти GraphQL; критерий здесь один и тот же: требования к латентности, трафику, отладке и эволюции контракта.

«Пусть пользователь подождёт тридцать секунд в синхронном запросе» обычно заканчивается зависшим UI и обрывами на балансировщике. Такие операции выносят в фон: `POST /reports` возвращает `202 Accepted` и `job_id`, задача уходит в брокер, результат приходит через polling или WebSocket-push.

Итог

Модуль 1 — про то, как **не привыкать к одному инструменту**. Протоколы, архитектурные стили, синхронные и асинхронные модели — это шкала вариантов, из которой выбирают под задачу. Без этой шкалы инженер решает по привычке; с ней — по критериям конкретной системы.